

TCP DUMP: THE BASICS

TCPdump library name? libpcap

- Specify the network interface: the first thing to decide which network interface to select for capturing packets:

tcpdump -i any = this selects all network interfaces
tcpdump -i interface name = that network interface will be selected

- How to save captured packets?

tcpdump -w filename select file extension as .pcap

- How to read captured packets from a file?

tcpdump -r filename

- How to limit no. of captured packets?

tcpdump -c count.

- How to not resolve ip address as domain name and port no. as their name?

tcpdump -n → used to not to resolve ip addresses

tcpdump -nn → used ^{to not to resolve} both ~~ip~~ ip address and port no.

- If you want more details about the packet as the output?

tcpdump -v

tcpdump -vv for more details

" -vvv " " "

`Tcpdump -i eth0 -c 50 -v` : captures 50 packets on a ethernet interface and displays them more detailed output

`Tcpdump -i wlan0 -w data.pcap` : captures packets from wifi interface and save them to `data.pcap` file. It will continue till user interrupt.

- Filtering by host: If you are interested in IP packets exchanged with your network printer or ^{specific} game server, Command: `host hostname or host IP.`

`sudo tcpdump host example.com -w http.pcap` : we capture all packets all the packets exchanged with `example.com` and save them to `http.pcap`. Capturing packets requires you to be root user or use `sudo`.

How to limit packets from a particular source IP address or hostname? = `src host ip or src host hostname`

How to limit packets sent to a specific destination? `dst host hostname or dst host ip`

- Filtering by port:

Command = `tcpdump port portnumber`

↓
will filter all the packets with the specified port number either the port number is in source port or destination port. But if you want to filter packets with the specified portnumber either as source port or destination port separately you can use:

`tcpdump src port portnumber`

`tcpdump dst port portnumber`

- Filtering by protocol:

Command: `tcpdump protocolname`

- Logical operators:

and: Captures packets where both conditions are true.

Ex: `tcpdump host 1.1.1.1 and tcp`

or: Captures packets where either one of the conditions is true.

Ex: `tcpdump udp or icmp`

not: Captures packets where the condition is not true.

Ex: `tcpdump not tcp`

- `tcpdump greater length`: Filters packets that have a length greater or equal to specified length.

`tcpdump smaller length`: Filters packets that have a length smaller or equal to specified length.

- Header bytes: It helps us to filter packets based on the contents of a header byte.

Using `pcap-filter` `tcpdump` allows you to refer the content of any bytes in the header using the syntax: `proto [expr:size]`

where `proto` refers to protocol

• `expr` refers to byte offset, where 0 refers to the first byte
size refers to no. of bytes that interest us.

tcp-syn (synchronize)
tcp-ack (acknowledge)
tcp-fin (finish)
tcp-rst (reset)

tcp-push (push)

Page No.	
Date:	

TCP

- How to filter packets using TCP flags?

Command: `tcp[tcpflags]`

`tcpdump "tcp[tcpflags] == tcp-syn"` : to capture tcp packet with only syn flag set while all the other are unset. Ex: SYN

`tcpdump "tcp[tcpflags] & tcp-syn != 0"` : to capture TCP packets with atleast the syn flag set. Ex: SYN, SYN-ACK

`tcpdump "tcp[tcpflags] & (tcp-syn | tcp-ack) != 0"` : to capture packets with at least the syn or ack flag sets. Ex: SYN, ACK, SYN-ACK

- Displaying packets: Packets can be displayed and Printed with many customizations:

- q : Quick output, print brief packet information
- e : Print link level header. Include MAC addr in output
- A : Show packet data in ASCII
- xx : Show packet data in hexadecimal format
- X : Show packet headers and data in hex and ASCII